Prolog Project

Title:Simple Library System

Team Members:

Rewan Reda Ragab 921230043

Shahd Mohamed Hussien 921230060

Abdelrahman yehia Saeed 921230081

Habiba Mohamed Bakr 921230036

Marvel Akram 921230103

Introduction:

This project implements a simple Library Management System using Prolog.

The goal of the project is to demonstrate how the concept of relations and logical rules can be used to represent knowledge and answer queries.

The system models students, books, authors, and categories. It also keeps track of borrowed books and allows the system to determine book availability, recommend books, and count borrowed books using logical inference.

Prolog was chosen because it is a logic programming language that is suitable for representing knowledge and building intelligent systems.

Artificial Intelligence concepts were applied through the use of Prolog's logical reasoning and inference mechanism. Instead of following step-by-step instructions like traditional programming, the system uses rules and pattern matching to automatically derive answers from the knowledge base. This allows the system to solve problems such as determining whether a book is available, counting borrowed books, and recommending books based on category.

AI tools were also used during the development process to assist in generating ideas, organizing the knowledge base, improving the structure of the rules, and explaining the logic behind the program. These tools helped speed up development and ensured that the system worked correctly.

Although the system is simple, it demonstrates how Artificial Intelligence techniques and logic programming can be used to solve real-world problems. The project can be expanded in the future by adding more books, users, borrowing rules, and smarter recommendation features.

Problem Description :

The problem addressed in this project is managing a small library system.

The system should be able to:

- Store information about students and books
- Store book authors and categories
- Track which student borrowed which book
- Check if a book is available
- Recommend books based on category
- Count borrowed books for each student

The system uses Prolog facts to represent knowledge and rules to perform reasoning and answer queries.

Knowledge Representation (Facts):

The system represents knowledge using Prolog facts. These facts describe the relationships between different entities in the system.

Examples include:

student(StudentName) → represents students in the system.

book(BookName) → represents books available in the library.

author(Book, Author) → shows the relationship between books and their authors.

category(Book, Category) → defines the category of each book.

borrowed(Student, Book) → represents which student borrowed which book.

These facts form the knowledge base of the system.

Rules:

The system also includes logical rules that allow Prolog to infer new information.

Important rules include:

available(Book)

Checks whether a book exists and is not currently borrowed.

books_by_author(Author, Book)

Returns books written by a specific author.

books_in_category(Category, Book)

Returns books that belong to a given category.

borrowed_count(Student, Count)

Counts how many books a student borrowed using the findall predicate.

can_borrow(Student)

Determines if a student can borrow more books (maximum 2).

recommend_book(Category, Book)

Recommends an available book from a given category.

author_book_count(Author, Count)

Counts how many books an author has using recursion.

Code

```
% ========================
% Simple Library System
% ========================

% -------- Facts --------

% Students
student(marvel).
student(abdelrahman).
student(shahd).
student(rawan).
student(habiba).

% Books
book(clean_code).
book(the_hobbit).
book(nineteen_eighty_four).
book(to_kill_a_mockingbird).
book(the_alchemist).
book(the_little_prince).
book(rich_dad_poor_dad).
book(atomic_habits).
book(sapiens).
book(harry_potter).
book(the_great_gatsby).
book(crime_and_punishment).

% Authors
author(clean_code, robert_martin).
author(the_hobbit, tolkien).
author(nineteen_eighty_four, george_orwell).
author(to_kill_a_mockingbird, harper_lee).
author(the_alchemist, paulo_coelho).
author(the_little_prince, saint_exupery).
author(rich_dad_poor_dad, robert_kiyosaki).
author(atomic_habits, james_clear).
author(sapiens, yuval_harari).
author(harry_potter, jk_rowling).
author(the_great_gatsby, fitzgerald).
author(crime_and_punishment, dostoevsky).
```

```
% Categories
category(clean_code, programming).
category(the_hobbit, fantasy).
category(nineteen_eighty_four, fiction).
category(to_kill_a_mockingbird, classic).
category(the_alchemist, fiction).
category(the_little_prince, fiction).
category(rich_dad_poor_dad, finance).
category(atomic_habits, self_help).
category(sapiens, history).
category(harry_potter, fantasy).
category(the_great_gatsby, classic).
category(crime_and_punishment, classic).

% Borrowed books
borrowed(marvel, the_hobbit).
borrowed(abdelrahman, clean_code).
borrowed(shahd, atomic_habits).
borrowed(rawan, the_alchemist).
borrowed(habiba, harry_potter).

% -------- Rules --------

% 1) Book is available if it exists and is not borrowed
available(Book) :-
    book(Book),
    not(borrowed(_, Book)).

% 2) Get books by a specific author
books_by_author(Author, Book) :-
    author(Book, Author).

% 3) Get books in a specific category
books_in_category(Category, Book) :-
    category(Book, Category).

% 4) Check if a student borrowed a specific book
has_borrowed(Student, Book) :-
    borrowed(Student, Book).

% 5) Count borrowed books for a student
borrowed_count(Student, Count) :-
    findall(Book, borrowed(Student, Book), Books),
    length(Books, Count).
```

```
    length(books, Count).

% 6) Student can borrow if borrowed less than 2 books
can_borrow(Student) :-
    student(Student),
    borrowed_count(Student, Count),
    Count < 2.

% 7) Recommend an available book from a category
recommend_book(Category, Book) :-
    category(Book, Category),
    available(Book).

% 8) Check if a borrowed book belongs to a category
borrowed_from_category(Student, Category) :-
    borrowed(Student, Book),
    category(Book, Category).

% 9) Recursive rule to count list elements
list_count([], 0).
list_count([_|T], Count) :-
    list_count(T, C1),
    Count is C1 + 1.

% 10) Count books by an author using recursion
author_book_count(Author, Count) :-
    findall(Book, author(Book, Author), List),
    list_count(List, Count).
```

Query and screen shoots

```
29 ?- consult('libraryyy.pl').
true.

30 ?- books_by_author(tolkien, Book).
Book = the_hobbit.

31 ?- books_in_category(fantasy, Book).
Book = the_hobbit .

32 ?- books_in_category(classic, Book).
Book = to_kill_a_mockingbird .

33 ?- borrowed_count(marvel, Count).
Count = 1.
```

```
35 ?- author_book_count(tolkien, Count).
Count = 1.

36 ?- available(Book).
Book = nineteen_eighty_four
```

Conclusion

This project implemented a simple library system using Prolog to demonstrate how logical programming can represent knowledge and relationships between different entities such as books, authors, categories, and borrowers. The system was built using facts to represent stored information and rules to describe logical relationships that allow the program to answer queries about book availability, borrowed books, and book recommendations.